

```
import os
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from matplotlib.image import imread
```

```
!pip install -U gdown
```

```
Requirement already satisfied: gdown in /usr/local/lib/python3.11/dist-packages (5.2.0)
Requirement already satisfied: beautifulsoup4 in /usr/local/lib/python3.11/dist-packages (from gdown) (4.13.4)
Requirement already satisfied: filelock in /usr/local/lib/python3.11/dist-packages (from gdown) (3.18.0)
Requirement already satisfied: requests[socks] in /usr/local/lib/python3.11/dist-packages (from gdown) (2.32.3)
Requirement already satisfied: tqdm in /usr/local/lib/python3.11/dist-packages (from gdown) (4.67.1)
Requirement already satisfied: soupsieve>1.2 in /usr/local/lib/python3.11/dist-packages (from beautifulsoup4->gdown) (2.
Requirement already satisfied: typing-extensions>=4.0.0 in /usr/local/lib/python3.11/dist-packages (from beautifulsoup4-
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.11/dist-packages (from requests[socks]
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.11/dist-packages (from requests[socks]->gdown) (3.
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.11/dist-packages (from requests[socks]->gdow
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.11/dist-packages (from requests[socks]->gdow
Requirement already satisfied: PySocks!=1.5.7,>=1.5.6 in /usr/local/lib/python3.11/dist-packages (from requests[socks]->
```

```
import gdown
```

```
# File ID
file_id = '1-Hx0D4jRYwP8LVhbalNgjYY4I-41Dg9a'
# Output name
output = 'cell_images_split.zip'

# Download from Google Drive
gdown.download(id=file_id, output=output, quiet=False)
```

```
Downloading...
From (original): https://drive.google.com/uc?id=1-Hx0D4jRYwP8LVhbalNgjYY4I-41Dg9a
From (redirected): https://drive.google.com/uc?id=1-Hx0D4jRYwP8LVhbalNgjYY4I-41Dg9a&confirm=t&uuiid=e4ff258f-0bfe-4427-85
To: /content/cell_images_split.zip
100%[██████████] 356M/356M [00:06<00:00, 57.6MB/s]
'cell_images_split.zip'
```

```
!unzip -q cell_images_split.zip
```

```
my_data_dir = '/content/cell_images/cell_images'
train_path = my_data_dir + '/train/'
test_path = my_data_dir + '/test/'
```

```
os.listdir(test_path)
```

```
['parasitized', 'uninfected']
```

```
os.listdir(train_path)
```

```
['parasitized', 'uninfected']
```

```
os.listdir(train_path+'/parasitized')[2]
```

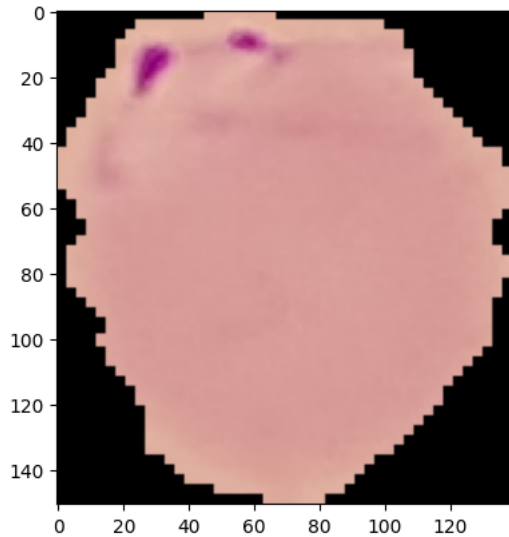
```
'C48P9thinF_IMG_20150721_160944_cell_231.png'
```

```
para_cell = train_path+'/parasitized'+'/C100P61ThinF_IMG_20150918_144104_cell_164.png'
```

```
para_img= imread(para_cell)
```

```
plt.imshow(para_img)
```

```
<matplotlib.image.AxesImage at 0x79bfdc962090>
```

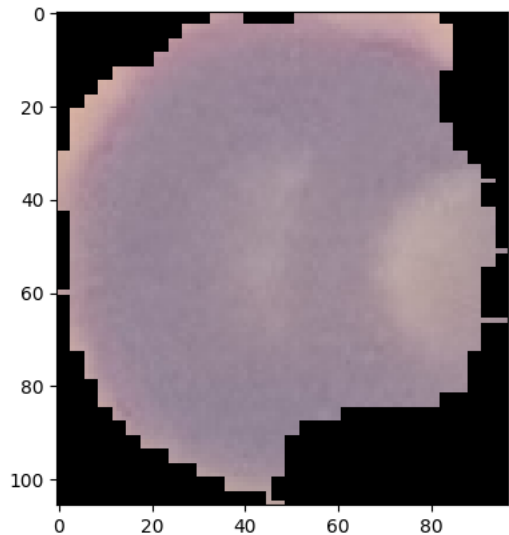


```
para_img.shape
```

```
(151, 139, 3)
```

```
uninfected_cell_path = train_path+'/uninfected/'+os.listdir(train_path+'/uninfected')[5]
uninfected_cell = imread(uninfected_cell_path)
plt.imshow(uninfected_cell)
```

```
<matplotlib.image.AxesImage at 0x79bfdc809590>
```



```
len(os.listdir(train_path+'/parasitized'))
```

```
12480
```

```
len(os.listdir(train_path+'/uninfected'))
```

```
12480
```

```
uninfected_cell.shape
```

```
(106, 97, 3)
```

```
para_img.shape
```

```
(151, 139, 3)
```

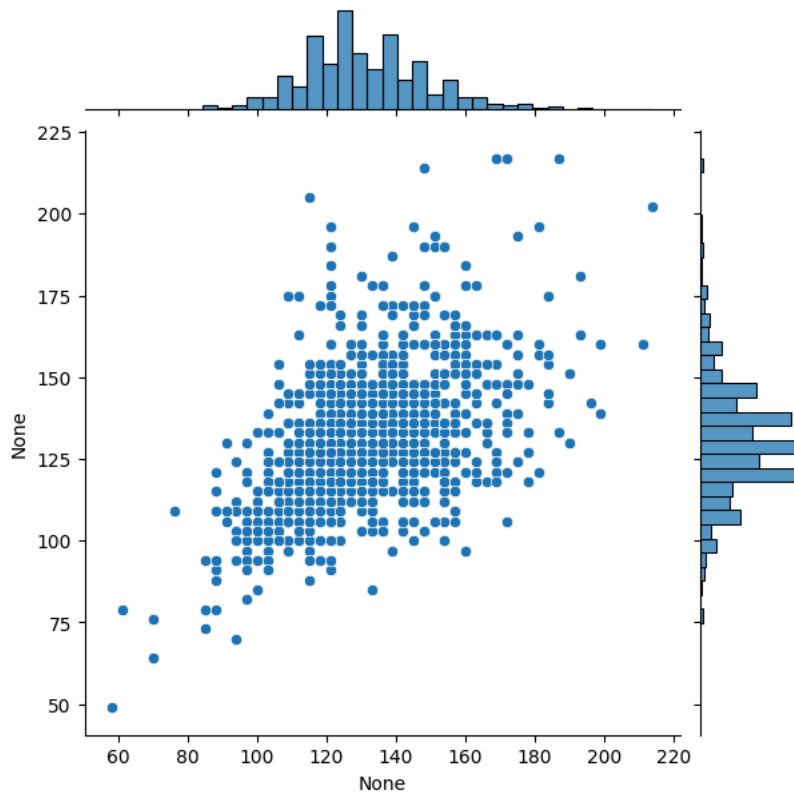
```
dim1 = []
dim2 = []
for image_filename in os.listdir(test_path+'/uninfected'):

    img = imread(test_path+'/uninfected/'+image_filename)
    d1,d2,colors = img.shape
```

```
dim1.append(d1)
dim2.append(d2)
```

```
sns.jointplot(x=dim1,y=dim2)
```

```
<seaborn.axisgrid.JointGrid at 0x79bfdc6acdd0>
```



```
np.mean(dim1)
```

```
np.float64(130.92538461538462)
```

```
np.mean(dim2)
```

```
np.float64(130.75)
```

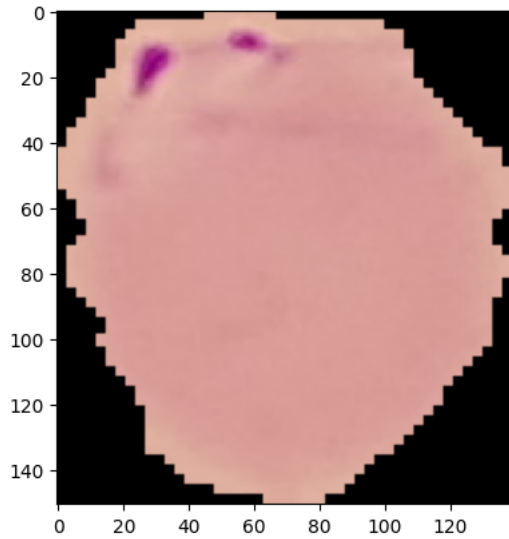
```
image_shape = (130,130,3)
```

```
from tensorflow.keras.preprocessing.image import ImageDataGenerator
```

```
image_gen = ImageDataGenerator(rotation_range=20,
                                width_shift_range=0.10,
                                height_shift_range=0.10,
                                rescale=1/255,
                                shear_range=0.1,
                                zoom_range=0.1,
                                horizontal_flip=True,
                                fill_mode='nearest'
                                )
```

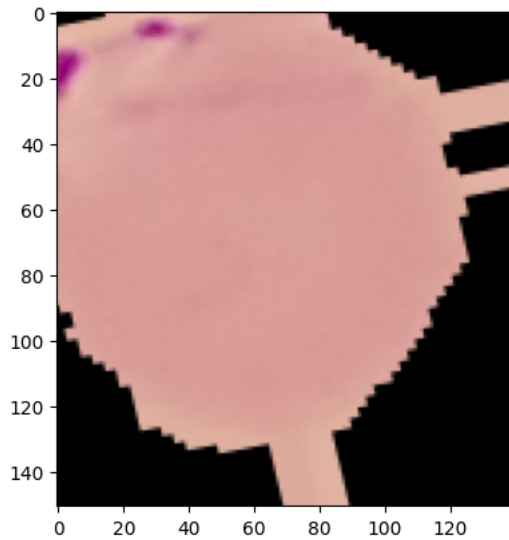
```
plt.imshow(para_img)
```

↗ `<matplotlib.image.AxesImage at 0x79bf76fa8110>`



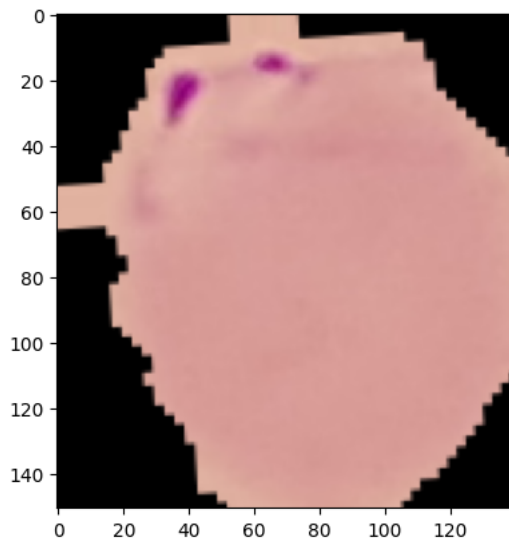
```
plt.imshow(image_gen.random_transform(para_img))
```

↗ `<matplotlib.image.AxesImage at 0x79bf76f88110>`



```
plt.imshow(image_gen.random_transform(para_img))
```

↗ `<matplotlib.image.AxesImage at 0x79bfdb798110>`



```
image_gen.flow_from_directory(train_path)
```

↗ Found 24958 images belonging to 2 classes.
`<keras.src.legacy.preprocessing.image.DirectoryIterator at 0x79bf76e3bb90>`

```
image_gen.flow_from_directory(test_path)
```

Found 2600 images belonging to 2 classes.
<keras.src.legacy.preprocessing.image.DirectoryIterator at 0x79bf76ee2450>

✓ Creating the Model VGG16

```
from tensorflow.keras.applications import VGG16
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout, GlobalAveragePooling2D
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.callbacks import EarlyStopping, ModelCheckpoint

# Load VGG16 base (without top)
vgg_base = VGG16(include_top=False, input_shape=image_shape, weights='imagenet')
vgg_base.trainable = False # Freeze all layers
```

```
# Build model on top of VGG16
model = Sequential()
model.add(vgg_base)
model.add(GlobalAveragePooling2D())
model.add(Dense(128, activation='relu'))
model.add(Dropout(0.5))
model.add(Dense(1, activation='sigmoid')) # Binary classification
```

```
# Compile model
model.compile(optimizer=Adam(learning_rate=0.0001), loss='binary_crossentropy', metrics=['accuracy'])

model.summary()
```

Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/vgg16/vgg16_weights_tf_dim_ordering_t
58889256/58889256 4s 0us/step
Model: "sequential"

Layer (type)	Output Shape	Param #
vgg16 (Functional)	(None, 4, 4, 512)	14,714,688
global_average_pooling2d (GlobalAveragePooling2D)	(None, 512)	0
dense (Dense)	(None, 128)	65,664
dropout (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 1)	129

Total params: 14,780,481 (56.38 MB)
Trainable params: 65,793 (257.00 KB)
Non-trainable params: 14,714,688 (56.13 MB)

```
early_stop = EarlyStopping(monitor='val_loss', patience=5, restore_best_weights=True)
```

```
checkpoint = ModelCheckpoint('best_vgg16_model.h5', monitor='val_loss', save_best_only=True, verbose=1)
```

```
batch_size = 32
```

```
train_image_gen = image_gen.flow_from_directory(train_path,
                                                target_size=image_shape[:2],
                                                color_mode='rgb',
                                                batch_size=batch_size,
                                                class_mode='binary')
```

Found 24958 images belonging to 2 classes.

```
test_image_gen = image_gen.flow_from_directory(test_path,
                                                target_size=image_shape[:2],
                                                color_mode='rgb',
                                                batch_size=batch_size,
                                                class_mode='binary',
                                                shuffle=False)
```

Found 2600 images belonging to 2 classes.

```
history = model.fit(
    train_image_gen,
    epochs=20,
```

```

validation_data=test_image_gen,
callbacks=[early_stop, checkpoint]
)

/usr/local/lib/python3.11/dist-packages/keras/src/trainers/data_adapters/py_dataset_adapter.py:121: UserWarning: Your
self._warn_if_super_not_called()
Epoch 1/20
780/780 ━━━━━━━━━━━ 0s 148ms/step - accuracy: 0.6645 - loss: 0.6136
Epoch 1: val_loss improved from inf to 0.41719, saving model to best_vgg16_model.h5
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This fil
780/780 ━━━━━━━━━━━ 139s 167ms/step - accuracy: 0.6646 - loss: 0.6135 - val_accuracy: 0.8404 - val_loss: 0.417
Epoch 2/20
780/780 ━━━━━━━━━━━ 0s 140ms/step - accuracy: 0.8514 - loss: 0.3895
Epoch 2: val_loss improved from 0.41719 to 0.31838, saving model to best_vgg16_model.h5
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This fil
780/780 ━━━━━━━━━━━ 121s 155ms/step - accuracy: 0.8514 - loss: 0.3895 - val_accuracy: 0.8731 - val_loss: 0.318
Epoch 3/20
780/780 ━━━━━━━━━━━ 0s 141ms/step - accuracy: 0.8791 - loss: 0.3175
Epoch 3: val_loss improved from 0.31838 to 0.28181, saving model to best_vgg16_model.h5
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This fil
780/780 ━━━━━━━━━━━ 121s 155ms/step - accuracy: 0.8791 - loss: 0.3175 - val_accuracy: 0.8904 - val_loss: 0.281
Epoch 4/20
780/780 ━━━━━━━━━━━ 0s 140ms/step - accuracy: 0.8961 - loss: 0.2761
Epoch 4: val_loss improved from 0.28181 to 0.25036, saving model to best_vgg16_model.h5
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This fil
780/780 ━━━━━━━━━━━ 121s 155ms/step - accuracy: 0.8961 - loss: 0.2761 - val_accuracy: 0.8977 - val_loss: 0.250
Epoch 5/20
780/780 ━━━━━━━━━━━ 0s 139ms/step - accuracy: 0.9027 - loss: 0.2505
Epoch 5: val_loss improved from 0.25036 to 0.23758, saving model to best_vgg16_model.h5
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This fil
780/780 ━━━━━━━━━━━ 121s 154ms/step - accuracy: 0.9027 - loss: 0.2505 - val_accuracy: 0.9112 - val_loss: 0.237
Epoch 6/20
780/780 ━━━━━━━━━━━ 0s 140ms/step - accuracy: 0.9081 - loss: 0.2422
Epoch 6: val_loss improved from 0.23758 to 0.22745, saving model to best_vgg16_model.h5
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This fil
780/780 ━━━━━━━━━━━ 121s 155ms/step - accuracy: 0.9081 - loss: 0.2422 - val_accuracy: 0.9162 - val_loss: 0.227
Epoch 7/20
780/780 ━━━━━━━━━━━ 0s 140ms/step - accuracy: 0.9132 - loss: 0.2278
Epoch 7: val_loss did not improve from 0.22745
780/780 ━━━━━━━━━━━ 121s 155ms/step - accuracy: 0.9132 - loss: 0.2278 - val_accuracy: 0.9046 - val_loss: 0.231
Epoch 8/20
780/780 ━━━━━━━━━━━ 0s 141ms/step - accuracy: 0.9158 - loss: 0.2228
Epoch 8: val_loss did not improve from 0.22745
780/780 ━━━━━━━━━━━ 122s 156ms/step - accuracy: 0.9158 - loss: 0.2228 - val_accuracy: 0.9073 - val_loss: 0.227
Epoch 9/20
780/780 ━━━━━━━━━━━ 0s 142ms/step - accuracy: 0.9171 - loss: 0.2201
Epoch 9: val_loss improved from 0.22745 to 0.22247, saving model to best_vgg16_model.h5
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This fil
780/780 ━━━━━━━━━━━ 123s 157ms/step - accuracy: 0.9171 - loss: 0.2201 - val_accuracy: 0.9123 - val_loss: 0.222
Epoch 10/20
780/780 ━━━━━━━━━━━ 0s 140ms/step - accuracy: 0.9188 - loss: 0.2165
Epoch 10: val_loss improved from 0.22247 to 0.20873, saving model to best_vgg16_model.h5
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This fil
780/780 ━━━━━━━━━━━ 121s 155ms/step - accuracy: 0.9188 - loss: 0.2165 - val_accuracy: 0.9173 - val_loss: 0.208
Epoch 11/20
780/780 ━━━━━━━━━━━ 0s 140ms/step - accuracy: 0.9176 - loss: 0.2134
Epoch 11: val_loss improved from 0.20873 to 0.20641, saving model to best_vgg16_model.h5
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This fil
780/780 ━━━━━━━━━━━ 121s 154ms/step - accuracy: 0.9176 - loss: 0.2134 - val_accuracy: 0.9204 - val_loss: 0.206
Epoch 12/20
780/780 ━━━━━━━━━━━ 0s 141ms/step - accuracy: 0.9206 - loss: 0.2141
Epoch 12: val_loss did not improve from 0.20641

```

```

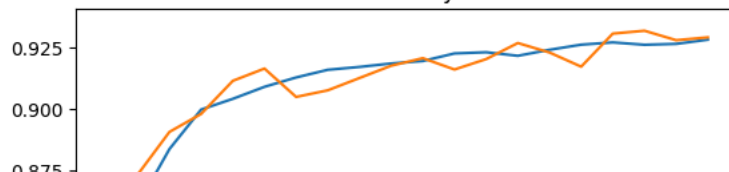
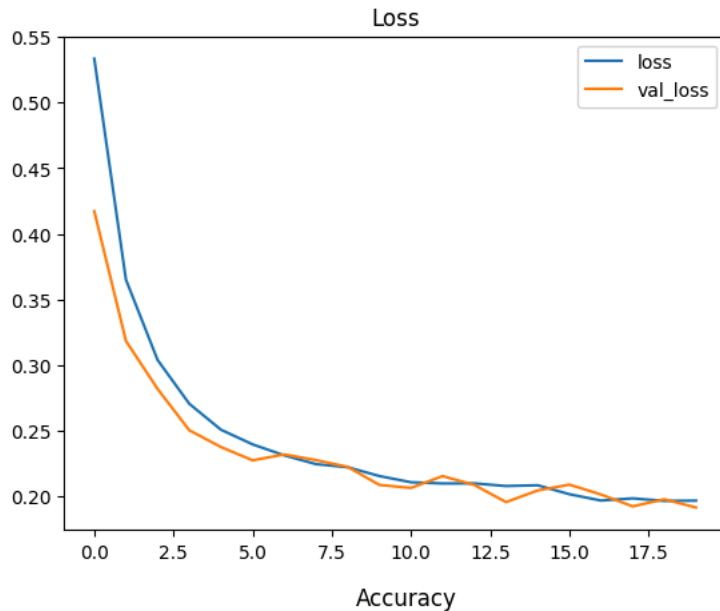
import pandas as pd
import matplotlib.pyplot as plt

losses = pd.DataFrame(history.history)

losses[['loss', 'val_loss']].plot(title='Loss')
losses[['accuracy', 'val_accuracy']].plot(title='Accuracy')

```

<Axes: title={'center': 'Accuracy'}>



```
# Evaluate
model.evaluate(test_image_gen)

# Predict probabilities
pred_probs = model.predict(test_image_gen)

# Use 0.80 threshold
predictions = pred_probs > 0.80

# Metrics
from sklearn.metrics import classification_report, confusion_matrix

print(classification_report(test_image_gen.classes, predictions.astype(int)))
print(confusion_matrix(test_image_gen.classes, predictions.astype(int)))
```

82/82 12s 139ms/step - accuracy: 0.9107 - loss: 0.1996
82/82 13s 145ms/step

	precision	recall	f1-score	support
0	0.85	0.96	0.90	1300
1	0.95	0.83	0.89	1300
accuracy			0.89	2600
macro avg	0.90	0.89	0.89	2600